INFORMATICS INSTITUTE OF TECHNOLOGY

UNIVERSITY OF WESTMINSTER

University of Westminster

# Client-Server Architectures

5COSC022C.2

## Coursework Report

Full Name - Onitha Bimsath Hettiarachchi

UoW Number - w2151901

IIT Number - 20232271

# Part 1: Service Architecture & Setup

## 1. Project & Application Configuration

### Question

In your report, explain the default lifecycle of a JAX-RS Resource class. Is a new instance instantiated for every incoming request, or does the runtime treat it as a singleton? Elaborate on how this architectural decision impacts the way you manage and synchronize your in-memory data structures to prevent data loss or race conditions.

### Answer

By default, JAX-RS resource classes are request-scoped. This means the JAX-RS runtime creates a new resource instance for each incoming HTTP request.

This is important because instance variables inside a resource class should not be used as the main storage for application data. If each request creates a new resource object, then storing rooms or sensors directly inside the resource object could cause data to be lost between requests.

To solve this, this project uses a shared DataStore class. The resources access the same data store using:

*DataStore.getInstance()*

This allows all resource classes to work with the same in-memory data.

Because several requests may access the same data at the same time, thread safety is needed. This project uses:

- ConcurrentHashMap for room and sensor storage.
- AtomicInteger for generating unique IDs.
- synchronized blocks for operations that update more than one collection.

For example, when a sensor is created, the API must add the sensor to the sensor collection and also add the sensor ID to the correct room. These two actions should happen safely together. Synchronization helps prevent partial updates and race conditions.

## 2. The "Discovery" Endpoint

**Question**

Why is the provision of "Hypermedia" links and navigation within responses considered a hallmark of advanced RESTful design? How does this approach benefit client developers compared to static documentation?

**Answer**

Hypermedia means that API responses include links to related resources. In this project, the discovery endpoint returns metadata and links to the main resource collections.

For example, the discovery response includes links such as:

```
{
  "links": {
    "self": "/api/v1",
    "rooms": "/api/v1/rooms",
    "sensors": "/api/v1/sensors"
  }
}
```

This helps clients understand where to go next without hard-coding every endpoint manually.

This approach is useful because:

- It makes the API easier to discover.
- It helps client developers navigate the API from a single starting point.
- It reduces dependency on static documentation.
- It allows the server to guide clients to available resources.

Static documentation can become outdated, but links returned directly by the API reflect the current API structure.

# Part 2: Room Management

## 1. Room Resource Implementation

### Question

When returning a list of rooms, what are the implications of returning only IDs versus returning the full room objects? Consider network bandwidth and client side processing.

### Answer

Returning only room IDs makes the response smaller. This reduces network bandwidth because the server sends less data.

Example:

*["room-1", "room-2"]*

However, this creates more work for the client. If the client needs the room name, capacity, or sensor IDs, it must send extra requests for each room. This increases client-side processing and can cause more network traffic overall.

Returning full room objects gives the client all important room information in one response.

Example:

```
[
  {
    "id": "room-1",
    "name": "Lecture Hall A",
    "capacity": 120,
    "sensorIds": ["sensor-1", "sensor-2"]
  }
]
```

This response is larger, but it is easier for clients because they do not need to make many extra requests. In this project, returning full room objects is suitable because the Room object is small and useful for client applications.

### 2. Room Deletion & Safety Logic

**Question**

Is the DELETE operation idempotent in your implementation? Provide a detailed justification by describing what happens if a client mistakenly sends the exact same DELETE request for a room multiple times.

**Answer**

Yes, the DELETE operation is idempotent because sending the same DELETE request multiple times results in the same final server state.

If a room exists and has no sensors, the first DELETE request removes the room and returns 204 No Content.

If the same DELETE request is sent again, the room is already deleted. The API returns 404 Not Found, but the final state is still the same: the room does not exist.

This means the response status may be different, but the server state is unchanged after repeated requests.

If the room still has sensors assigned to it, the API blocks the deletion and returns 409 Conflict. This prevents orphaned sensor data.

## Part 3: Sensor Operations & Linking

### 1. Sensor Resource & Integrity

**Question**

We explicitly use the @Consumes(MediaType.APPLICATION_JSON) annotation on the POST method. Explain the technical consequences if a client attempts to send data in a different format, such as text/plain or application/xml. How does JAX-RS handle this mismatch?

**Answer**

The @Consumes(MediaType.APPLICATION_JSON) annotation tells JAX-RS that the method accepts JSON request bodies.

If a client sends data using another content type such as text/plain or application/xml, JAX-RS will not call the resource method. Instead, the request is rejected because the content type does not match what the method accepts.

The API returns:

*HTTP 415 Unsupported Media Type*

This helps protect the API from receiving unsupported data formats. It also makes the API contract clear to clients: POST requests must send JSON.

## 2. Filtered Retrieval & Search

**Question**

You implemented this filtering using @QueryParam. Contrast this with an alternative design where the type is part of the URL path, for example /api/v1/sensors/type/CO2. Why is the query parameter approach generally considered superior for filtering and searching collections?

**Answer**

Using a query parameter is better for filtering a collection.

This design:

*GET /api/v1/sensors?type=CO2*

means the client is asking for the sensors collection, but filtered by type.

A path design such as:

*GET /api/v1/sensors/type/CO2*

makes type look like a separate resource. This is less clear because filtering is not really a separate resource. It is a condition applied to an existing collection.

Query parameters are also easier to extend. For example, more filters can be added later:

*GET /api/v1/sensors?type=CO2&status=ACTIVE*

This makes the API more flexible and easier for clients to use.

# Part 4: Deep Nesting with Sub-Resources

## 1. The Sub-Resource Locator Pattern

### Question

Discuss the architectural benefits of the Sub-Resource Locator pattern. How does delegating logic to separate classes help manage complexity in large APIs compared to defining every nested path, for example sensors/{id}/readings/{rid}, in one massive controller class?

### Answer

A sub-resource locator is a JAX-RS method that has a @Path annotation but does not directly use an HTTP method annotation such as @GET or @POST.

In this project, SensorResource delegates reading-related requests to SensorReadingResource.

The nested path is:

*/api/v1/sensors/{sensorId}/readings*

This design is useful because the sensor logic and reading logic are separated into different classes.

The benefits are:

- SensorResource focuses on sensor operations.
- SensorReadingResource focuses on sensor reading operations.
- The code is easier to read.
- The code is easier to maintain.
- The API can grow without creating one very large controller class.
- Parent sensor validation can happen before the readings resource handles the request.

This keeps the project structure cleaner and makes each class responsible for one part of the API.

## 2. Historical Data Management

### Question

A successful POST to a reading must trigger an update to the currentValue field on the corresponding parent Sensor object to ensure data consistency across the API.

**Answer**

When a new reading is posted to a sensor, the API stores that reading in the sensor's historical reading list.

However, the parent Sensor object also has a currentValue field. This field should always show the latest sensor value.

For example, when the client sends:

*POST /api/v1/sensors/sensor-1/readings*

with this body:

```
{
  "value": 23.7
}
```

the API creates a new SensorReading and also updates the parent sensor's currentValue to 23.7.

This is important because the API should stay consistent. If a client later calls:

*GET /api/v1/sensors/sensor-1*

the sensor should show the latest current value.

Without this update, the readings list would contain the new value, but the sensor object could still show an old value.

# Part 5: Advanced Error Handling, Exception Mapping & Logging

## 1. Resource Conflict (409)

**Question**

Attempting to delete a Room that still has Sensors assigned to it should return an HTTP 409 Conflict with a JSON body explaining that the room is currently occupied by active hardware.

**Answer**

This project uses RoomNotEmptyException for this situation.

When a client tries to delete a room that still has sensors, the API blocks the request. This prevents orphaned sensor data.

The exception is mapped to:

*HTTP 409 Conflict*

This status code is suitable because the request conflicts with the current state of the room.

Example response:

```
{
  "errorMessage": "Room room-1 still has 2 sensor(s). Remove them first.",
  "errorCode": 409,
  "documentation": "https://developer.smartcampus.example/docs/errors#room-not-empty"
}
```

## 2. Dependency Validation (422 Unprocessable Entity)

**Question**

Why is HTTP 422 often considered more semantically accurate than a standard 404 when the issue is a missing reference inside a valid JSON payload?

**Answer**

A 404 Not Found means the requested URL does not exist.

In this case, the URL exists:

*POST /api/v1/sensors*

The problem is inside the JSON body. The client may send a roomId that does not exist.

Example:

```
{
  "type": "CO2",
```

*"status": "ACTIVE",*

*"currentValue": 400.0,*

*"roomId": "room-999"*

*}*

The JSON is valid and the endpoint exists, but the server cannot process the request because room-999 is not a valid room.

For this reason, 422 Unprocessable Entity is more accurate. It means the server understood the request, but the data inside the request cannot be processed according to the API's rules.

## 3. State Constraint (403 Forbidden)

**Question**

A sensor currently marked with the status MAINTENANCE is physically disconnected and cannot accept new readings. Map this to an HTTP 403 Forbidden status when a POST reading is attempted.

**Answer**

This project uses SensorUnavailableException for this case.

If a sensor is in MAINTENANCE status, it should not accept new readings. The endpoint exists, and the request may be valid, but the sensor's current state does not allow the operation.

The exception is mapped to:

*HTTP 403 Forbidden*

Example response:

*{*

*"errorMessage": "Sensor sensor-4 is currently under maintenance. Readings cannot be recorded.",*

*"errorCode": 403,*

*"documentation": "https://developer.smartcampus.example/docs/errors#sensor-unavailable"*

*}*

This makes the error clear to the client.

## 4. The Global Safety Net (500)

**Question**

From a cybersecurity standpoint, explain the risks associated with exposing internal Java stack traces to external API consumers. What specific information could an attacker gather from such a trace?

**Answer**

Exposing Java stack traces to clients is a security risk.

A stack trace can reveal internal details such as:

- Package names.
- Class names.
- Method names.
- File names.
- Line numbers.
- Framework details.
- Server paths.
- Internal application flow.

An attacker could use this information to understand how the application is built and look for weak points. For example, if the stack trace reveals framework versions or internal class names, the attacker may search for known vulnerabilities or target specific parts of the system.

To avoid this, this project uses a catch-all exception mapper:

*ExceptionMapper<Throwable>*

This mapper catches unexpected errors and returns a safe JSON response.

Example:

*{*

*"errorMessage": "An unexpected internal server error occurred.",*

*"errorCode": 500,*

*"documentation": "https://developer.smartcampus.example/docs/errors#internal-server-error"*

*}*

The full error is logged on the server side, but it is not shown to the client.

## 5. API Request & Response Logging Filters

**Question**

Why is it advantageous to use JAX-RS filters for cross-cutting concerns like logging, rather than manually inserting Logger.info() statements inside every single resource method?

**Answer**

Logging is a cross-cutting concern because it applies to all API endpoints, not just one method.

Using JAX-RS filters is better than writing Logger.info() inside every resource method because:

- It avoids repeated code.
- It logs all requests and responses in one place.
- It keeps resource methods focused on business logic.
- It makes the code easier to maintain.
- If the log format needs to change, only one filter class needs to be updated.

This project uses a logging filter that implements:

*ContainerRequestFilter*

*ContainerResponseFilter*

The request filter logs the HTTP method and request URI.

The response filter logs the HTTP status code.

This helps with debugging, testing, and observing how the API behaves during Postman or curl testing.